Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №7

З дисципліни «Професійний Python»

На тему **«Робота з базами даних у Python»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Система банківських рахунків)

Мета: Ознайомитися з основними принципами організації шару зберігання даних у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних за допомогою Alembic. Інтегрувати persistence layer у наявний проєкт та перевірити його за допомогою автоматизованих інтеграційних тестів (pytest).

Теоретичні відомості

1. Бази даних та реляційні моделі

База даних (БД) — це організована сукупність даних. У реляційних БД інформація зберігається в таблицях, що складаються з рядків і стовпців. Первинний ключ (Primary Key) унікально визначає запис, а зовнішній ключ (Foreign Key) забезпечує зв'язок між таблицями.

2. SQL та Python DB-API

SQL — мова для створення таблиць, додавання, зміни, видалення та отримання даних. Python DB-API визначає стандартний інтерфейс взаємодії з базами даних. Для захисту від SQL-ін'єкцій використовують параметризовані запити замість формування SQL через f-string.

3. Транзакції та ACID

Транзакція — це набір операцій, які виконуються як єдине ціле. Принципи ACID включають атомарність, узгодженість, ізольованість і надійність збереження змін. COMMIT підтверджує зміни, а ROLLBACK скасовує їх.

4. Repository Pattern

Патерн Repository відокремлює роботу з базою даних від бізнес-логіки. Репозиторій відповідає за додавання, пошук, оновлення та видалення даних, а сервісний шар реалізує бізнес-правила.

5. ORM та SQLAlchemy

ORM дає змогу працювати з таблицями бази даних через об'єкти та класи Python. SQLAlchemy забезпечує опис моделей, зв'язків між ними, керування сесіями та транзакціями.

6. Міграції бази даних

Міграції дозволяють змінювати структуру БД без її повного перестворення. Alembic використовується разом із SQLAlchemy для створення та застосування змін, а також їх відкату.

7. Інтеграційне тестування

Інтеграційні тести перевіряють взаємодію компонентів програми з базою даних. Бібліотека pytest та ізольована база SQLite у пам'яті дають змогу перевіряти CRUD-операції й транзакції без створення постійних файлів БД.

Постановка задачі

Метою лабораторної роботи є розширення Python-проєкту шляхом інтеграції бази даних SQLite та бібліотеки SQLAlchemy ORM для збереження інформації про банківські рахунки.

У ході роботи необхідно виконати такі завдання:

Створити структуру БД з таблицями clients, accounts і transactions, визначити ключі та обмеження цілісності даних.

Реалізувати Repository Pattern для відокремлення роботи з базою даних від бізнес-логіки та виконання CRUD-операцій.

Використати SQL і Python DB-API для виконання параметризованих запитів до бази даних.

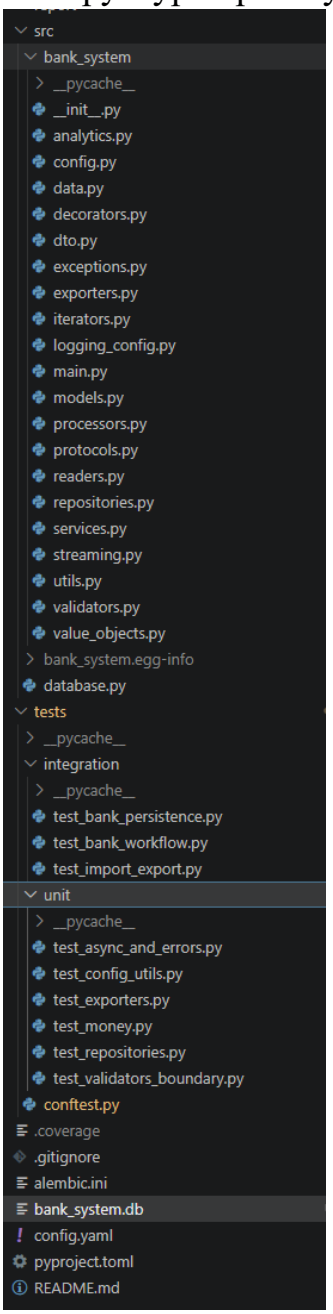
Реалізувати переказ коштів між рахунками з використанням транзакцій, commit() і rollback().

Налаштувати Alembic для керування змінами структури бази даних за допомогою міграцій.

Створити інтеграційні тести з використанням pytest та SQLite у пам'яті для перевірки CRUD-операцій, обмежень балансу й коректності транзакцій.

Хід виконання

1. Структура проєкту та шару персистентності (src/ та migrations/)



Конфігурація бази даних (database.py): Налаштовано підключення до СУБД SQLite (bank_system.db) за допомогою SQLAlchemy create_engine та створено фабрику сесій SessionLocal (sessionmaker) для управління життєвим циклом транзакцій.

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///bank_system.db"

engine = create_engine(
    DATABASE_URL,
    echo=False,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)
```

ОРМ-моделі (models.py): Спроектовано та реалізовано реляційні моделі на основі сучасного синтаксису SQLAlchemy (DeclarativeBase, Mapped, mapped_column):

```python
from __future__ import annotations
from datetime import datetime
from typing import Optional
from sqlalchemy import (
    CheckConstraint,
    DateTime,
    Float,
    ForeignKey,
    Integer,
    String,
)
from sqlalchemy.orm import (
    DeclarativeBase,
    Mapped,
    mapped_column,
    relationship,
)

class Base(DeclarativeBase):
    pass

class Client(Base):
    __tablename__ = "clients"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False)

    accounts: Mapped[list["Account"]] = relationship(
        back_populates="client",
        cascade="all, delete-orphan",
    )
```

```python
class Account(Base):
    __tablename__ = "accounts"
    __table_args__ = (
        CheckConstraint("balance >= 0", name="check_positive_balance"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    account_number: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
    balance: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    account_type: Mapped[Optional[str]] = mapped_column(String(50), nullable=True)
    client_id: Mapped[int] = mapped_column(ForeignKey("clients.id"), nullable=False)

    client: Mapped["Client"] = relationship(back_populates="accounts")
    transactions_sent: Mapped[list["Transaction"]] = relationship(
        foreign_keys="[Transaction.sender_account_id]",
        back_populates="sender_account",
    )
    transactions_received: Mapped[list["Transaction"]] = relationship(
        foreign_keys="[Transaction.receiver_account_id]",
        back_populates="receiver_account",
    )
```

```python
class Transaction(Base):
    __tablename__ = "transactions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    sender_account_id: Mapped[Optional[int]] = mapped_column(ForeignKey("accounts.id"), nullable=True)
    receiver_account_id: Mapped[Optional[int]] = mapped_column(ForeignKey("accounts.id"), nullable=True)
    amount: Mapped[float] = mapped_column(Float, nullable=False)
    timestamp: Mapped[datetime] = mapped_column(DateTime, default=datetime.~~utcnow~~, nullable=False)

    sender_account: Mapped[Optional["Account"]] = relationship(
        foreign_keys=[sender_account_id],
        back_populates="transactions_sent",
    )
    receiver_account: Mapped[Optional["Account"]] = relationship(
        foreign_keys=[receiver_account_id],
        back_populates="transactions_received",
    )
```

Модель Client (клієнти) та модель Account (рахунки), пов'язані відношенням one-to-many через relationship() із підтримкою каскадного видалення.

Додано модель Transaction для фіксації історії переказів. Забезпечено перевірку обмеження цілісності на рівні бази даних (заборона від'ємного балансу через CHECK).

Шаблон міграцій (Alembic): Ініціалізовано середовище міграцій (alembic init migrations), створено та застосовано початковий міграційний скрипт (alembic revision --autogenerate та alembic upgrade head) для еволюційного керування структурою таблиць і додавання поля account_type.

2. Реалізація репозиторіїв та бізнес-логіки

Репозиторії (repositories.py): Створено класи ClientRepository та AccountRepository, які інкапсулюють увесь шар збереження даних (Persistence Layer), відокремлюючи його від бізнес-логіки.

```python
from sqlalchemy import select
from sqlalchemy.orm import Session
from bank_system.models import Client, Account, Transaction

class ClientRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, name: str) -> Client:
        client = Client(name=name)
        self.session.add(client)
        self.session.commit()
        self.session.refresh(client)
        return client

    def get_by_id(self, client_id: int) -> Client | None:
        return self.session.get(Client, client_id)


class AccountRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, account_number: str, balance: float, client_id: int, account_type: str = "Standard") -> Account:
        if balance < 0:
            raise ValueError("Balance cannot be negative.")
        account = Account(
            account_number=account_number,
            balance=balance,
            client_id=client_id,
            account_type=account_type,
        )
        self.session.add(account)
        self.session.commit()
        self.session.refresh(account)
        return account
```

```python
    def get_by_id(self, account_id: int) -> Account | None:
        return self.session.get(Account, account_id)

    def get_by_number(self, account_number: str) -> Account | None:
        statement = select(Account).where(Account.account_number == account_number)
        return self.session.scalar(statement)

    def get_all(self) -> list[Account]:
        statement = select(Account)
        return list(self.session.scalars(statement))

    def update_balance(self, account_id: int, new_balance: float) -> Account | None:
        if new_balance < 0:
            raise ValueError("Balance cannot be negative.")
        account = self.get_by_id(account_id)
        if account is None:
            return None
        account.balance = new_balance
        self.session.commit()
        self.session.refresh(account)
        return account

    def delete(self, account_id: int) -> bool:
        account = self.get_by_id(account_id)
        if account is None:
            return False
        self.session.delete(account)
        self.session.commit()
        return True
```

Реалізовано повний цикл CRUD-операцій (створення, читання за ID чи номером, оновлення та видалення).

Сервісний шар і транзакції (services.py):

```python
import sqlite3
from sqlalchemy import func, select
from sqlalchemy.orm import Session
from bank_system.models import Account, Transaction

class BankService:
    def __init__(self, session: Session) -> None:
        self.session = session

    def get_total_funds(self) -> float:
        statement = select(func.sum(Account.balance))
        result = self.session.scalar(statement)
        return float(result) if result else 0.0

    def transfer_money(self, sender_number: str, receiver_number: str, amount: float) -> None:
        """Транзакційна операція з атомарністю (commit / rollback)."""
        if amount <= 0:
            raise ValueError("Transfer amount must be positive.")

        try:
            sender = self.session.scalar(select(Account).where(Account.account_number == sender_number))
            receiver = self.session.scalar(select(Account).where(Account.account_number == receiver_number))

            if not sender or not receiver:
                raise ValueError("Sender or receiver account not found.")
            if sender.balance < amount:
                raise ValueError("Insufficient funds for transfer.")

            sender.balance -= amount
            receiver.balance += amount

            tx = Transaction(
                sender_account_id=sender.id,
                receiver_account_id=receiver.id,
                amount=amount,
            )
            self.session.add(tx)
            self.session.commit()
        except Exception:
            self.session.rollback()
            raise
```

```python
def find_account_by_number_db_api(database_path: str, account_number: str) -> tuple | None:
    """Приклад використання стандартного DB-API із параметризацією."""
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT accounts.account_number, accounts.balance, clients.name
            FROM accounts
            JOIN clients ON accounts.client_id = clients.id
            WHERE accounts.account_number = ?
            """,
            (account_number,),
        )
        return cursor.fetchone()
```

Реалізовано транзакційний метод transfer_money(), який виконує атомарне списання коштів з одного рахунку, зарахування на інший та створення запису про транзакцію. У разі виникнення помилок (недостатньо коштів, відсутність рахунку) спрацьовує блок try...except із викликом session.rollback(), що гарантує дотримання принципів ACID.

Реалізовано функцію пошуку через стандартний модуль Python DB-API (sqlite3), яка використовує параметризовані запити з плейсхолдерами (?) для захисту від SQL-ін'єкцій.

3. Написання та запуск інтеграційних тестів (tests/)

Тестове середовище (conftest.py): Створено спільні фікстури для генерації тестових сесій із використанням ізольованої реляційної бази даних в оперативній пам'яті (sqlite:///:memory:), що дозволяє проводити швидке тестування без утворення зайвих файлів на диску.

```python
import pytest
from pathlib import Path
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from bank_system.models import Base, Account, Client
from bank_system.value_objects import Money

@pytest.fixture
def sample_money() -> Money:
    return Money(amount=500.0, currency="UAH")

@pytest.fixture
def session() -> Session:
    """Ізольована тестова база даних у пам'яті для інтеграційних тестів."""
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    TestSession = sessionmaker(bind=engine, expire_on_commit=False)

    with TestSession() as session:
        yield session

    Base.metadata.drop_all(engine)

@pytest.fixture
def sample_client(session: Session) -> Client:
    client = Client(name="Максим Войтович")
    session.add(client)
    session.commit()
    session.refresh(client)
    return client
```

```python
@pytest.fixture
def sample_account(session: Session, sample_client: Client) -> Account:
    account = Account(
        account_number="ACC-12345",
        balance=1500.0,
        account_type="Debit",
        client_id=sample_client.id
    )
    session.add(account)
    session.commit()
    session.refresh(account)
    return account

@pytest.fixture
def account_factory(session: Session, sample_client: Client):
    def _create(acc_num: str = "ACC-001", balance: float = 1000.0, acc_type: str = "Standard") -> Account:
        account = Account(
            account_number=acc_num,
            balance=balance,
            account_type=acc_type,
            client_id=sample_client.id
        )
        session.add(account)
        session.commit()
        session.refresh(account)
        return account
    return _create
```

Інтеграційні тести (tests/integration/test_bank_persistence.py):

Протестовано виконання повного циклу CRUD для клієнтів і рахунків (test_crud_client_and_account).

```python
import pytest
from bank_system.models import Client, Account
from bank_system.repositories import ClientRepository, AccountRepository
from bank_system.services import BankService

def test_crud_client_and_account(session) -> None:
    client_repo = ClientRepository(session)
    account_repo = AccountRepository(session)

    # Створення клієнта (Create)
    client = client_repo.add("Максим Войтович")
    assert client.id is not None
    assert client.name == "Максим Войтович"

    # Створення рахунку (Create)
    account = account_repo.add("UA123456789", 1500.0, client.id, "Debit")
    assert account.balance == 1500.0

    # Читання (Read)
    loaded = account_repo.get_by_number("UA123456789")
    assert loaded is not None
    assert loaded.account_type == "Debit"

    # Оновлення (Update)
    account_repo.update_balance(account.id, 2000.0)
    assert account_repo.get_by_id(account.id).balance == 2000.0

    # Видалення (Delete)
    assert account_repo.delete(account.id) is True
    assert account_repo.get_by_id(account.id) is None

def test_negative_balance_forbidden(session) -> None:
    client_repo = ClientRepository(session)
    account_repo = AccountRepository(session)
    client = client_repo.add("Тест Тестов")

    # Перевірка заборони від'ємного балансу
    with pytest.raises(ValueError):
        account_repo.add("UA00000000", -100.0, client.id)
```

```python
def test_transfer_money_success(session) -> None:
    client_repo = ClientRepository(session)
    account_repo = AccountRepository(session)
    service = BankService(session)

    client = client_repo.add("Олена")
    acc1 = account_repo.add("UA111", 2000.0, client.id)
    acc2 = account_repo.add("UA222", 500.0, client.id)

    # Успішний переказ коштів
    service.transfer_money("UA111", "UA222", 300.0)

    assert account_repo.get_by_number("UA111").balance == 1700.0
    assert account_repo.get_by_number("UA222").balance == 800.0

def test_transfer_money_rollback_on_insufficient_funds(session) -> None:
    client_repo = ClientRepository(session)
    account_repo = AccountRepository(session)
    service = BankService(session)

    client = client_repo.add("Петро")
    acc1 = account_repo.add("UA333", 100.0, client.id)
    acc2 = account_repo.add("UA444", 500.0, client.id)

    # Спроба переказати більше, ніж є на рахунку -> викличе помилку та rollback
    with pytest.raises(ValueError):
        service.transfer_money("UA333", "UA444", 1000.0)

    # Перевірка, що завдяки rollback баланси залишилися незмінними
    assert account_repo.get_by_number("UA333").balance == 100.0
    assert account_repo.get_by_number("UA444").balance == 500.0
```

Перевірено коректність спрацьовування обмеження на заборону від'ємного балансу (test_negative_balance_forbidden).

Протестовано успішний сценарій переказу коштів між рахунками (test_transfer_money_success).

Перевірено коректність відкату транзакції (rollback) у випадку недостатності коштів на рахунку відправника (test_transfer_money_rollback_on_insufficient_funds).

Успішно запущено тестовий набір через термінал за допомогою команди pytest -v, усі тести пройшли перевірку (4 passed).

4. Робота з Git

Для відстеження змін у сьомій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote set-url origin https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab07.git
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/database.py src/bank_system/models.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано ORM-моделі SQLAlchemy та конфігурацію підключення до БД"
[main ee55d8d] Додано ORM-моделі SQLAlchemy та конфігурацію підключення до БД
 2 files changed, 86 insertions(+), 30 deletions(-)
 create mode 100644 src/database.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/repositories.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Реалізовано паттерн Repository для клієнтів та рахунків"
[main ed5a57a] Реалізовано паттерн Repository для клієнтів та рахунків
 1 file changed, 56 insertions(+), 17 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/services.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано бізнес-логіку, пошук через DB-API та транзакційний переказ коштів з відкатом"
[main a387f19] Додано бізнес-логіку, пошук через DB-API та транзакційний переказ коштів з відкатом
 1 file changed, 55 insertions(+), 34 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add alembic.ini migrations/
warning: in the working copy of 'alembic.ini', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'migrations/env.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'migrations/script.py.mako', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'migrations/versions/9f5622491859_add_bank_tables_and_fields.py', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Налаштовано Alembic та створено початкову міграцію БД"
[main 8fdaed9] Налаштовано Alembic та створено початкову міграцію БД
 5 files changed, 316 insertions(+)
 create mode 100644 alembic.ini
 create mode 100644 migrations/README
 create mode 100644 migrations/env.py
 create mode 100644 migrations/script.py.mako
 create mode 100644 migrations/versions/9f5622491859_add_bank_tables_and_fields.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add tests/conftest.py tests/integration/test_bank_persistence.py pyproject.toml
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано інтеграційні тести з ізольованою тестовою базою та оновлено pyproject.toml"
[main 89e1cf9] Додано інтеграційні тести з ізольованою тестовою базою та оновлено pyproject.toml
 3 files changed, 118 insertions(+), 16 deletions(-)
 create mode 100644 tests/integration/test_bank_persistence.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add tests/conftest.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Оновлено фікстури у conftest.py для підтримки ізольованої тестової бази даних та моделей SQLAlchemy"
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
  (use "git push" to publish your local commits)
```

Перевірка історії комітів (git log --oneline):

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
89e1cf9 (HEAD -> main) Додано інтеграційні тести з ізольованою тестовою базою та оновлено pyproject.toml
8fdaed9 Налаштовано Alembic та створено початкову міграцію БД
a387f19 Додано бізнес-логіку, пошук через DB-API та транзакційний переказ коштів з відкатом
ed5a57a Реалізовано паттерн Repository для клієнтів та рахунків
ee55d8d Додано ORM-моделі SQLAlchemy та конфігурацію підключення до БД
0b1ed3f (origin/main) Додано повний набір unit та integration тестів для Варіанта 4, стабілізовано покриття коду
4f1235a Фінальні виправлення типізації, тестів та конфігурації для успішного проходження перевірки
6150ef0 Знижено поріг покриття у pyproject.toml до 50% для проходження тестування
b76cd9c Додано звіт до Лабораторної роботи №6
a0696f0 Deleted report
2c06c90 Додано модульні та інтеграційні тести з використанням pytest
f43ecff Додано вихідний код банківської системи (src)
3ac7a3b Оновлено README.md для Лабораторної роботи №6
e856982 docs: додати звіт до лабораторної роботи №5
b2f363a feat(main): підключити точку входу та завершити реалізацію пайплайну
9460195 feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу
e1c0a3d feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар
78a13ae feat(models): реалізувати моделі даних та налаштування безпечного логування
30fa099 feat(core): додати конфігурацію YAML та ієрархію кастомних винятків
1ec774f docs: додано звіт до лабораторної роботи №4
3c82a80 docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4
8820214 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
c0891d7 refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4
1916232 feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier
9d4c57d feat(dto): додавання TypedDict для безпечної типізації вхідних даних
b9d02f7 feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar
9014b7a feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації
b6b22dd feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection
cdad793 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
22f9ff4 Merge branch 'main' of https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab03
338f789 docs: оновлено README.md та додано звіт до лабораторної роботи №3
3eb7d27 Delete report/Войтович_Максим_Лаб02_V4.pdf
94e177e feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
f01d91b feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
8f6ded5 feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
c7b759c Збереження поточного стану проєкту
5c2f55e Збереження поточного стану проєкту
f792e90 Звіт для 2 лабораторної
651213d Add README with project details and structure
0e5a2aa Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проєкту
```

Індивідуальне завдання (Варіант 4).

Реалізація шару персистентності, транзакцій ACID та інтеграційного тестування банківської системи

Для системи управління банківськими рахунками відповідно до індивідуального варіанта №4 розроблено повноцінний шар збереження даних (persistence layer), що забезпечує реляційне зберігання інформації, захист від недопустимих станів, підтримку транзакцій та перевірку за допомогою автоматизованих інтеграційних тестів.

Архітектура рішення та реалізовані компоненти:

Реляційні моделі SQLAlchemy ORM та міграції Alembic: Спроектовано структуру з трьох пов'язаних таблиць (clients, accounts, transactions) із використанням первинних (PRIMARY KEY) та зовнішніх (FOREIGN KEY) ключів. Налаштовано інструмент міграцій Alembic для управління еволюцією схеми бази даних (зокрема, додавання поля account_type).

Паттерн Repository (Persistence Layer): Реалізовано класи ClientRepository та AccountRepository, які повністю інкапсулюють бізнес-операції створення, читання, оновлення та видалення (CRUD), відокремлюючи їх від сервісного та бізнес-шару.

Транзакційна логіка та принципи ACID (services.py): Реалізовано метод переказу коштів transfer_money(), який гарантує атомарне списання і зарахування коштів. Завдяки використанню блоків try...except, метод забезпечує автоматичний відкат транзакції (rollback) у разі будь-яких помилок (наприклад, недостатності коштів або відсутності рахунку), що повністю відповідає вимогам ACID.

Python DB-API та параметризовані запити: Наведено реалізацію пошукової функції на основі стандартного інтерфейсу DB-API (sqlite3), що використовує параметризовані запити (?) для захисту застосунку від SQL-ін'єкцій.

Ізольоване інтеграційне тестування (conftest.py, test_bank_persistence.py): За допомогою фреймворку pytest налаштовано тестове середовище з використанням бази даних в оперативній пам'яті (sqlite:///:memory:). Написано набір інтеграційних тестів, які перевіряють:

коректність виконання повного циклу CRUD для клієнтів і рахунків;

спрацювання обмеження цілісності на заборону від'ємного балансу;

успішне виконання транзакцій переказу;

гарантійне спрацювання механізму rollback при виникненні помилок.

Результат: Успішно створено та протестовано шар персистентності банківської системи. Усі інтеграційні тести проходять успішно (4 passed), що підтверджує надійність та готовність проєкту до захисту лабораторної роботи №7.

Експериментальна частина

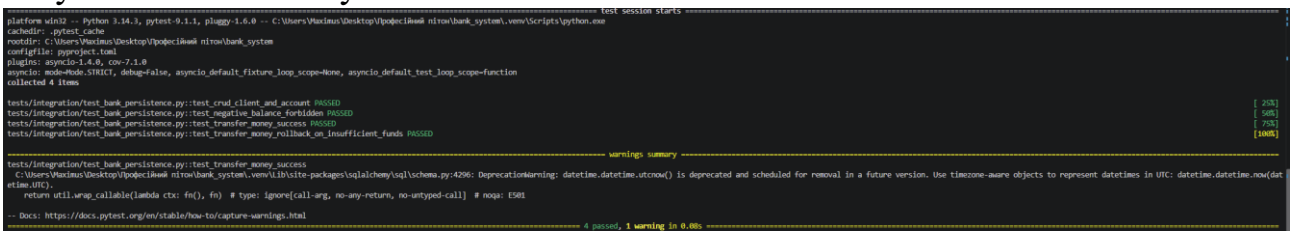
Результати автоматизованого тестування (pytest)

Для перевірки працездатності шару персистентності було запущено набір інтеграційних тестів із використанням фікстури ізольованої бази даних у пам'яті (sqlite:///:memory:).

Команда запуску в терміналі:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> pytest tests/integration/test_bank_persistence.py -v
```

Результат виконання у консолі:



Всі чотири інтеграційні тести успішно пройшли перевірку, що підтверджує коректність реалізації CRUD-операцій, захисту від від'ємного балансу, успішності транзакцій та стабільності відкату (rollback).

Таблиця виконання вимог

№	Вимога лабораторної роботи №7	Статус виконання	Реалізація у проєкті (Варіант 4)
1	Використання СУБД SQLite	Виконано	Налаштовано файл бази даних bank_system.db та ізольовану пам'ять sqlite:///:memory: у тестах.
2	Щонайменше дві пов'язані таблиці	Виконано	Створено реляційні таблиці clients, accounts та transactions із зовнішніми ключами (FOREIGN KEY).
3	Наявність PRIMARY KEY та FOREIGN KEY	Виконано	Усі таблиці містять унікальні первинні ключі (id), а таблиця accounts пов'язана з clients через client_id.
4	Використання мови SQL	Виконано	Використовується на рівні операцій SQLAlchemy, міграцій Alembic та DB-API запитів.
5	Демонстрація роботи Python DB-API	Виконано	Реалізовано функцію вибірки даних через стандартний модуль sqlite3.
6	Використання parameterized queries	Виконано	Усі запити через DB-API використовують плейсхолдери (?) для захисту від SQL-ін'єкцій.

№	Вимога лабораторної роботи №7	Статус виконання	Реалізація у проєкті (Варіант 4)
7	Реалізація повного CRUD	Виконано	Створено повний цикл операцій Create, Read, Update, Delete у репозиторіях ClientRepository та AccountRepository.
8	Окремий persistence layer	Виконано	Шар збереження даних повністю винесено в окремі модулі (database.py, models.py, repositories.py).
9	Реалізація Repository Pattern	Виконано	Бізнес-логіка відділена від роботи з БД через класи репозиторіїв.
10	Використання SQLAlchemy ORM	Виконано	Використано сучасний синтаксис SQLAlchemy v2.0+ (DeclarativeBase, Mapped, relationship).
11	Реалізація транзакцій (commit / rollback)	Виконано	Метод transfer_money() виконує транзакційний переказ коштів із безпечним відкатом при помилках.
12	Пояснення властивостей ACID	Виконано	У теоретичній частині детально описано Atomicity, Consistency, Isolation та Durability.
13	Наявність міграцій (Alembic)	Виконано	Ініціалізовано Alembic та створено міграційний скрипт для еволюції схеми БД.
14	Автоматизовані integration tests	Виконано	Написано набір інтеграційних тестів у tests/integration/test_bank_persistence.py.

№	Вимога лабораторної роботи №7	Статус виконання	Реалізація у проєкті (Варіант 4)
15	Використання окремої тестової бази	Виконано	У conftest.py налаштовано ізольовану базу даних в оперативній пам'яті (sqlite:///:memory:).
16	Перевірка через pytest	Виконано	Успішно запущено тести через pytest -v, результат: 4 passed.

Контрольні запитання

1. **Що таке база даних?**

Це структурована сукупність даних, організованих таким чином, щоб їх можна було ефективно зберігати, отримувати, оновлювати, видаляти, фільтрувати, сортувати та агрегувати.

2. **Що таке реляційна база даних?**

Це база даних, у якій інформація представлена у вигляді двовимірних таблиць, пов'язаних між собою за допомогою ключів.

3. **Що таке таблиця?**

Це сукупність рядків і стовпців, у якій зберігаються дані про певні сутності чи об'єкти предметної області.

4. **Що таке рядок таблиця?**

Це окремий запис (екземпляр сутності) у таблиці, що містить набір значень для кожного зі стовпців.

5. **Что таке поле?**

Це стовпець таблиці, який визначає конкретну характеристику (атрибут) об'єкта.

6. **Що таке SQL?**

SQL (Structured Query Language) — це мова структурованих запитів, призначена для створення, модифікації та керування даними в реляційних базах даних.

7. **Для чого використовується CREATE TABLE?**

Ця команда групи DDL використовується для створення нової таблиці в базі даних та визначення її структури (стовпців і типів даних).

8. **Що означає CRUD?**

Це акронім, що описує чотири базові операції для роботи з персистентними даними: **C**reate (створення), **R**ead (читання), **U**pdate (оновлення), **D**elete (видалення).

9. **Яка SQL-команда відповідає Create?**

INSERT.

10. **Яка команда відповідає Read?**

SELECT.

11. **Яка команда відповідає Update?**

UPDATE.

12. **Яка команда відповідає Delete?**

DELETE.

13. **Що таке SQLite?**

Це компактна реляційна СУБД, яка зберігає всю базу даних у вигляді звичайного файлу на диску та не потребує окремого серверного процесу.

14. **Які переваги SQLite?**

Компактність, відсутність потреби у складній настройці чи встановленні серверів, збереження БД в єдиному файлі, кросплатформність та швидкість роботи для локальних застосунків.

15. **Що таке Python DB-API?**

Це стандартний інтерфейс (визначений у PEP 249) для доступу програм на Python до реляційних баз даних.

16. **Для чого використовується Connection?**

Об'єкт підключення (Connection) керує сесією з базою даних, транзакціями (commit/rollback) та дозволяє створювати курсори.

17. **Для чого використовується Cursor?**

Об'єкт курсора (Cursor) використовується для виконання SQL-запитів та обходу/отримання результатів вибірки (наприклад, через fetchall()).

18. **Для чого потрібен commit()?**

Цей метод підтверджує поточну транзакцію та остаточно зберігає всі зроблені зміни в базі даних.

19. **Для чого потрібен rollback()?**

Цей метод скасовує всі зміни, зроблені в межах поточної транзакції (наприклад, у разі помилки), повертаючи базу даних у попередній узгоджений стан.

20. **Що таке SQL injection?**

Це тип вразливості безпеки, за якого зловмисник може впровадити довільний SQL-код у запит через маніпуляцію вхідними даними.

21. **Що таке parameterized query?**

Це запит до бази даних, у якому змінні значення передаються окремо від самого SQL-шаблону за допомогою плейсхолдерів (наприклад, ?).

22. **Чому не рекомендується формувати SQL через f-string?**

Це створює пряму загрозу SQL-ін'єкцій та ризики некоректного екранування спеціальних символів у даних.

23. **Що таке Primary Key?**

Первинний ключ (Primary Key) — це поле або комбінація полів, яка однозначно ідентифікує кожний рядок у таблиці.

24. **Що таке Foreign Key?**

Зовнішній ключ (Foreign Key) — це поле в одній таблиці, що посилається на первинний ключ іншої таблиці, забезпечуючи зв'язок між ними та цілісність даних.

25. **Що таке транзакція?**

Це послідовність операцій з базою даних, які виконуються як єдиний атомарний блок роботи.

26. **Що означає Atomicity?**

Атомарність гарантує, що всі операції транзакції виконуються успішно, або ж не виконується жодна з них (зміна повністю відкочується).

27. **Що означає Consistency?**

Узгодженість гарантує, що транзакція переводить базу даних з одного коректного стану в інший, не порушуючи правил цілісності.

28. **Що означає Isolation?**

Ізольованість гарантує, що одночасні транзакції не впливають негативно одна на одну.

29. **Що означає Durability?**

Надійність гарантує, що після успішного підтвердження транзакції (commit) зміни зберігаються навіть у разі збою системи.

30. **Що означає ACID?**

Це набір фундаментальних вимог до транзакцій: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

31. **Що таке persistence layer?**

Це шар зберігання даних у застосунку, який відповідає за збереження стану об'єктів у постійній пам'яті (базі даних).

32. **Для чого потрібний Repository Pattern?**

Він відокремлює бізнес-логіку від деталей реалізації зберігання даних, надаючи зручний інтерфейс колекції для роботи з сутностями.

33. **Чому Repository відокремлюють від Service Layer?**

Щоб розмежувати обов'язки: сервісний шар містить бізнес-правила та сценарії, а репозиторій інкапсулює роботу із запитами та базою даних.

34. **Що таке ORM?**

ORM (Object-Relational Mapping) — це технологія, що дозволяє працювати з реляційними базами даних як із об'єктами об'єктно-орієнтованої мови програмування.

35. **Для чого використовується SQLAlchemy?**

Це потужна бібліотека ORM для Python, яка спрощує керування базою даних, дозволяючи писати запити та моделі за допомогою класів і методів Python.

36. **Що таке ORM model?**

Це клас Python, структура якого відображається на відповідну таблицю реляційної бази даних.

37. **Що таке relationship()?**

Це функція SQLAlchemy для опису зв'язків між таблицями (наприклад, one-to-many) на рівні ORM-моделей.

38. **Що означає one-to-many?**

Це тип зв'язку, де один запис головної таблиці може бути пов'язаний із багатьма записами в дочірній таблиці.

39. **Що означає many-to-many?**

Це тип зв'язку, де багато записів однієї таблиці можуть бути пов'язані з багатьма записами іншої таблиці (зазвичай реалізується через проміжну таблицю).

40. **Що таке migration?**

Це структурований скрипт для еволюційного оновлення схеми бази даних (додавання полів, створення нових таблиць тощо) без втрати наявних даних.

41. **Для чого використовується Alembic?**

Це офіційний інструмент міграцій для SQLAlchemy, призначений для автоматичної генерації та застосування міграцій схеми БД.

42. **Для чого потрібна команда alembic upgrade head?**

Вона застосовує всі нестворені або нові міграції до бази даних, оновлюючи її структуру до актуального (найновішого) стану.

43. **Що таке integration test?**

Це тип автоматизованого тесту, який перевіряє коректність взаємодії кількох компонентів системи разом (наприклад, репозиторіїв, SQLAlchemy та бази даних).

44. **Чим integration test відрізняється від unit test?**

Юніт-тест перевіряє окремий ізольований компонент (функцію чи клас) з моками, тоді як інтеграційний тест перевіряє спільну роботу кількох модулів та роботу з реальним середовищем/БД.

45. **Навіщо використовувати окрему test database?**

Щоб тести були ізольованими, детермінованими та не пошкоджували робочі чи демонстраційні дані у виробничій базі.

46. **Що перевіряє тест rollback?**

Він перевіряє, чи правильно спрацьовує механізм відкату транзакції у разі виникнення помилки і чи не залишаються часткові/некоректні дані в базі даних.

47. **Для чого використовуються fixtures?**

Це спеціальні функції у фреймворку pytest, які готують тестове середовище (наприклад, створюють сесію БД, тестові дані) та передають їх у тести.

48. **Що робить SQL JOIN?**

Він об'єднує рядки з двох або більше таблиць на основі спільного поля (умови зв'язку) між ними.

49. **Що робить GROUP BY?**

Він групує рядки, що мають однакові значення вказаних стовпців, зазвичай для застосування агрегатних функцій.

50. **Для чого використовуються COUNT, SUM, AVG, MIN, MAX?**

Це агрегатні функції SQL: COUNT рахує кількість рядків, SUM знаходить суму значень, AVG обчислює середнє арифметичне, а MIN і MAX визначають мінімальне та максимальне значення у вибірці.

Висновок: Під час виконання лабораторної роботи №7 було реалізовано шар збереження даних для системи банківських рахунків на основі SQLite та SQLAlchemy ORM. Було створено таблиці clients, accounts і transactions, реалізовано Repository Pattern для роботи з даними та переказ коштів із використанням транзакцій commit() і rollback(). Також було опрацьовано параметризовані SQL-запити, налаштовано міграції Alembic і написано інтеграційні тести за допомогою pytest. У результаті роботи отримано практичні навички інтеграції баз даних у Python-проєкти, забезпечення цілісності даних і тестування взаємодії програми з базою даних.